

Estudo de Caso:

Criação, Implementação e Segurança de um Sistema de gerenciamento de tarefas pessoais

Wagner Cipriano da Silva

Data entrega: 14/06

Entrega:

Documentar o processo de desenvolvimento, implementação e segurança do sistema.

Relatório final (PDF), com no máximo **20 páginas** (conforme email enviado pela professora em Mon, Jun 8, 9:13 PM), descrevendo o projeto, as etapas realizadas (com prints das telas mais importantes), as ferramentas utilizadas, os resultados alcançados e as lições aprendidas.

Sumário

1- Planejamento e Definição de Requisitos.....	2
Requisitos não funcionais [RF].....	2
Requisitos não funcionais [RNF].....	2
Diagramas UML.....	2
Ameaças de Segurança e Medidas de Mitigação.....	3
Arquitetura do Sistema.....	3
2- Desenvolvimento do Sistema.....	4
Containerização.....	5
Melhorias.....	5
Logs.....	5
3- Integração contínua e testes automatizados.....	6
Controle de versão.....	6
Pipeline CI/CD.....	8
4- Análise Estática de Código (SAST).....	11
5- Análise Dinâmica de Segurança (DAST).....	13
6- Entrega Contínua (CD).....	15
7- Feedback e Monitoramento.....	18
Containers Docker.....	18
Prometheus.....	18
Grafana.....	19
Implementações.....	19
8- Conclusão.....	20
Ferramentas.....	20
Resultados alcançados.....	20
Lições aprendidas.....	20
Trabalhos futuros.....	20

1- Planejamento e Definição de Requisitos

Requisitos não funcionais [RF]

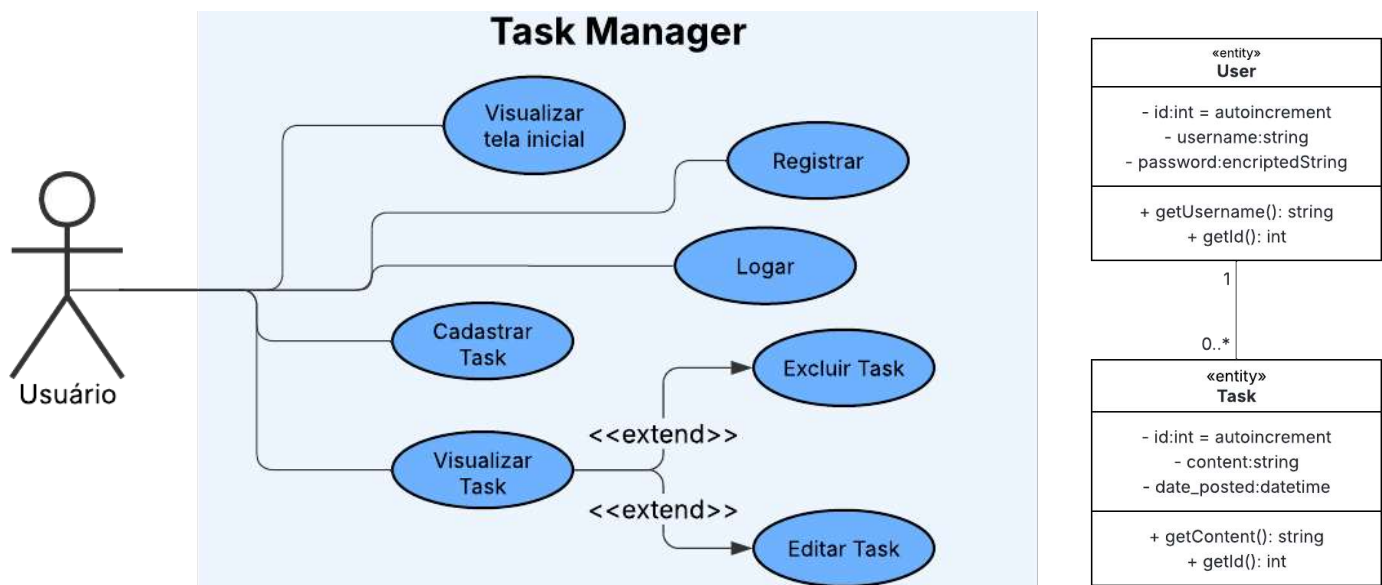
ID	Descrição	Prioridade
RF-01	Cadastrar usuário	Alta
RF-02	Autenticação e autorização	Alta
RF-03	Perfil (visualizar e alterar dados da conta)	Média
RF-04	Ver página home com as opções (logout, ver tarefas, etc)	Alta
RF-05	CRUD tarefas (create, read, update, delete)	Alta
RF-06	Pesquisar tarefas	Baixa
RF-07	Login com o google	Baixa

Requisitos não funcionais [RNF]

ID	Descrição	Prioridade
RNF-01	O site deve ser compatível com os principais navegadores	Alta
RNF-02	O site deverá ser responsivo permitindo a visualização em um celular de forma adequada	Alta
RNF-03	Acessibilidade (leitor de tela)	Média
RNF-04	Disponibilidade 99%	Média
RNF-05	Gerar logs de todas as atividades via syslog, bem como sucesso ou erro de autenticação	Alta

Diagramas UML

A seguir os diagramas de Casos de Uso e Diagrama de Classes.



Ameaças de Segurança e Medidas de Mitigação

Roubo de credenciais

Não armazenar senhas, apenas o hash da senha.

Utilizar hash criptográfico com o algoritmo SHA-256, para armazenamento de senhas mais seguro.

Adicionar um salt ao processo de hashing para garantir a unicidade das senhas, aumentar sua complexidade sem aumentar os requisitos do usuário e mitigar ataques como os de tabelas hash.

Proteção de dados pessoais

Para garantir cumprimento dos regulamentos vigentes, principalmente a LGDP, o sistema terá controle de acesso aos dados. Somente o proprietário poderá acessar os seus dados.

DDoS

Para proteção contra os ataques de negação de serviço serão utilizadas estratégias de monitoramento de tráfego de rede. A partir da criação de uma linha de base com o padrão normal de tráfego, a rede será monitorada para identificar mais facilmente os sintomas de um ataque DDoS. A partir da comparação com a linha de base, podem ser identificados problemas como baixo desempenho da rede, falhas intermitentes na web, etc.

Logs de violação de segurança

Os eventos violação de segurança que serão registrados via syslog para monitoramento:

- Falha de autenticação (datahora, ip, usuário)
- Page not found [404]
- Permission denied [403]

Arquitetura do Sistema

Containerização

Docker

Backend

Lógica de negócios e a interação com o banco de dados

Linguagem Python, framework Flask, Banco de dados SQLite.

Frontend

HTML, CSS, JavaScript.

2- Desenvolvimento do Sistema

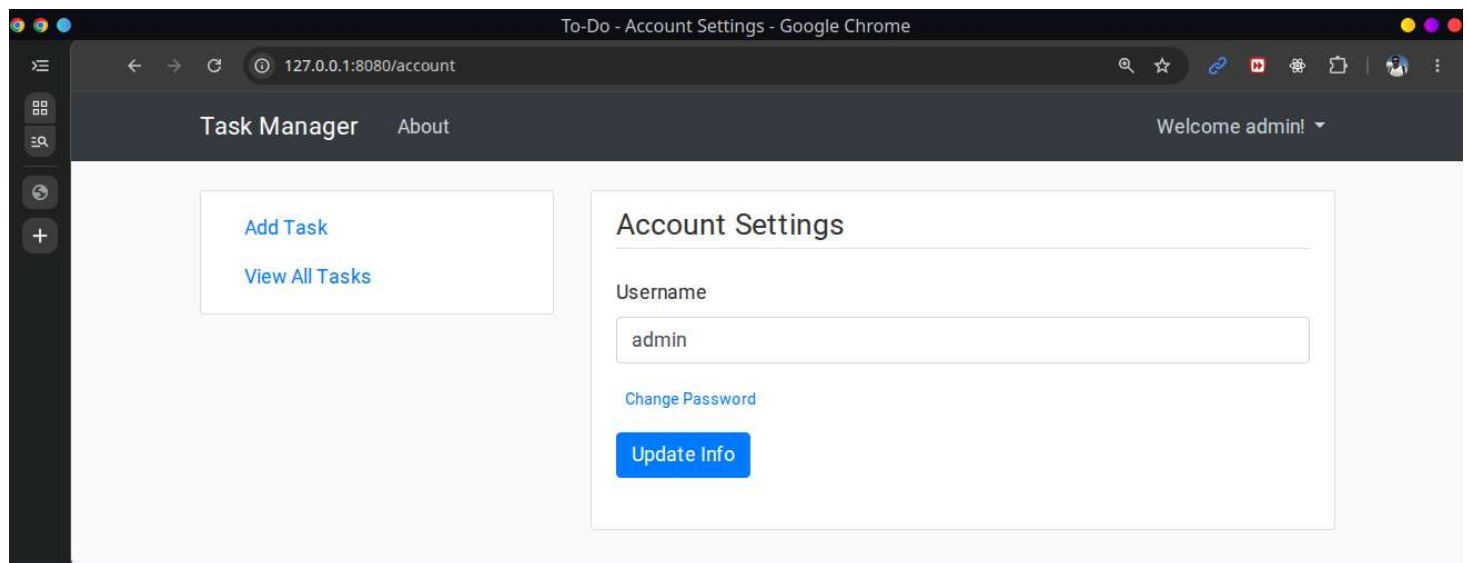
A partir do código base disponibilizado ([Task-Manager-using-Flask.git](https://github.com/wcipriano/task-manager-using-flask)) foi feito o fork para o repositório abaixo e em seguida foi feito o clone na máquina local dev:

- <https://github.com/wcipriano/case-study-hdb-task-manager/>

Todos os arquivos do projeto estão disponíveis neste repositório, caso algum print não seja suficiente, os arquivos podem ser acessados no repo acima, pois não é possível colocar print de tudo.

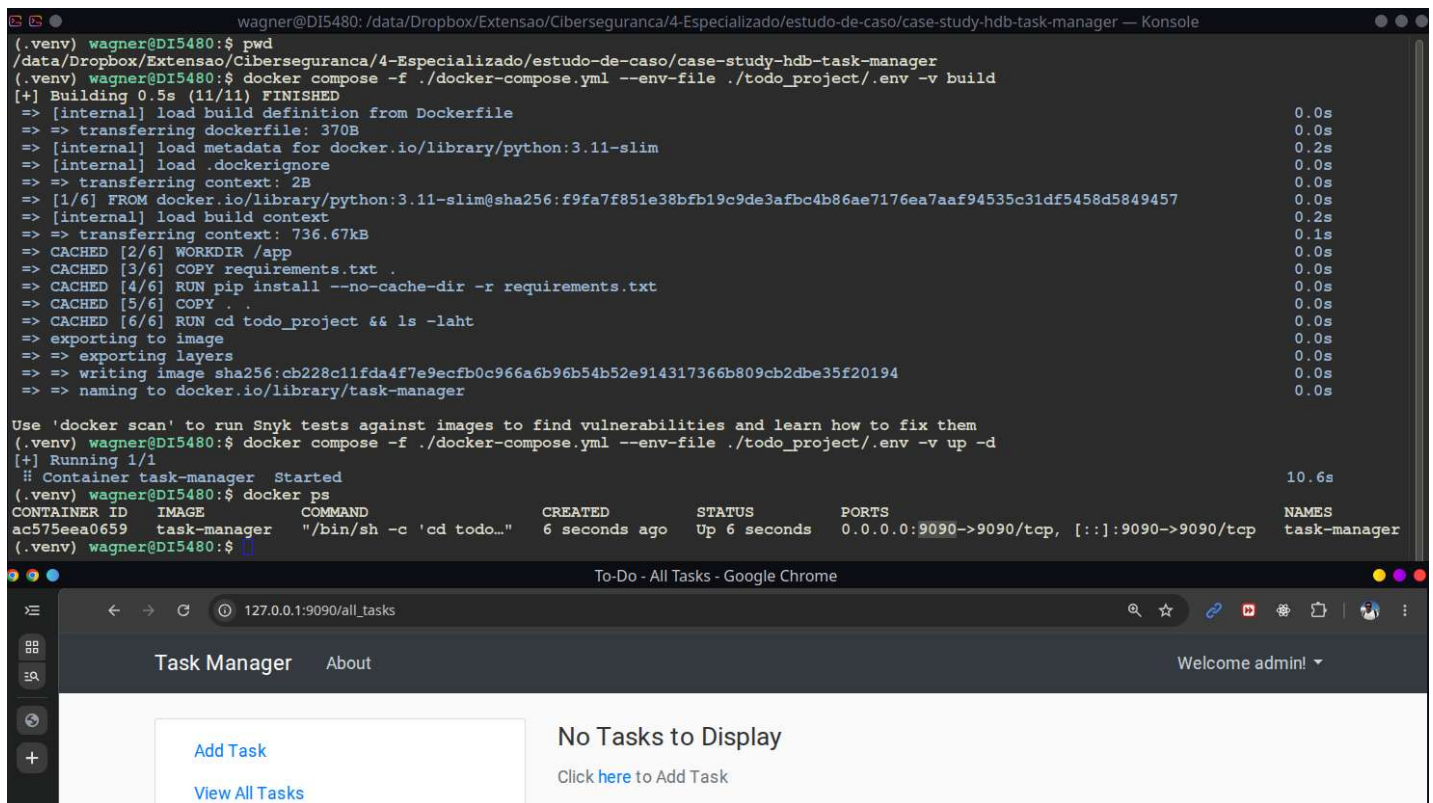
Após o setup, configuração do python e do virtual env, definição das variáveis de ambiente, algumas correções de dependências quebradas e a inicialização do banco de dados (SQLite), o sistema foi iniciado com sucesso, conforme prints abaixo:

```
wagner@DI5480: /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manage
(.venv) wagner@DI5480:$ pwd
/data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager/todo_project
(.venv) wagner@DI5480:$ cat ./env
FLASK_APP="todo_app"
FLASK_ENV=development
FLASK_DEBUG=1
FLASK_RUN_PORT="8080"
(.venv) wagner@DI5480:$ flask run
* Serving Flask app 'todo_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
* Running on http://127.0.0.1:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 895-922-883
127.0.0.1 - - [11/Jun/2026 18:37:14] "GET /login HTTP/1.1" 302 -
127.0.0.1 - - [11/Jun/2026 18:37:14] "GET /all_tasks HTTP/1.1" 200 -
```



Containerização

Em seguida, o sistema foi preparado para execução em ambiente containerizado com a utilização Docker. Foram criados os arquivos docker-compose.yml e Dockerfile.



```
wagner@DI5480: /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager — Konsole
(.venv) wagner@DI5480:~$ pwd
/data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager
(.venv) wagner@DI5480:~$ docker compose -f ./docker-compose.yml --env-file ./todo_project/.env -v build
[+] Building 0.5s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 370B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.11-slim@sha256:f9fa7f851e38bfb19c9de3afbc4b86ae7176ea7aaf94535c31df5458d5849457
=> [internal] load build context
=> => transferring context: 736.67kB
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt
=> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/6] COPY .
=> CACHED [6/6] RUN cd todo_project && ls -laht
=> exporting to image
=> => exporting layers
=> => writing image sha256:cb228c11fda4f7e9ecfb0c966a6b96b54b52e914317366b809cb2dbe35f20194
=> => naming to docker.io/library/task-manager

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
(.venv) wagner@DI5480:~$ docker compose -f ./docker-compose.yml --env-file ./todo_project/.env -v up -d
[+] Running 1/1
  # Container task-manager Started
(.venv) wagner@DI5480:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
ac575eea0659   task-manager   "/bin/sh -c 'cd todo..." 6 seconds ago   Up 6 seconds   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   task-manager
(.venv) wagner@DI5480:~$
```

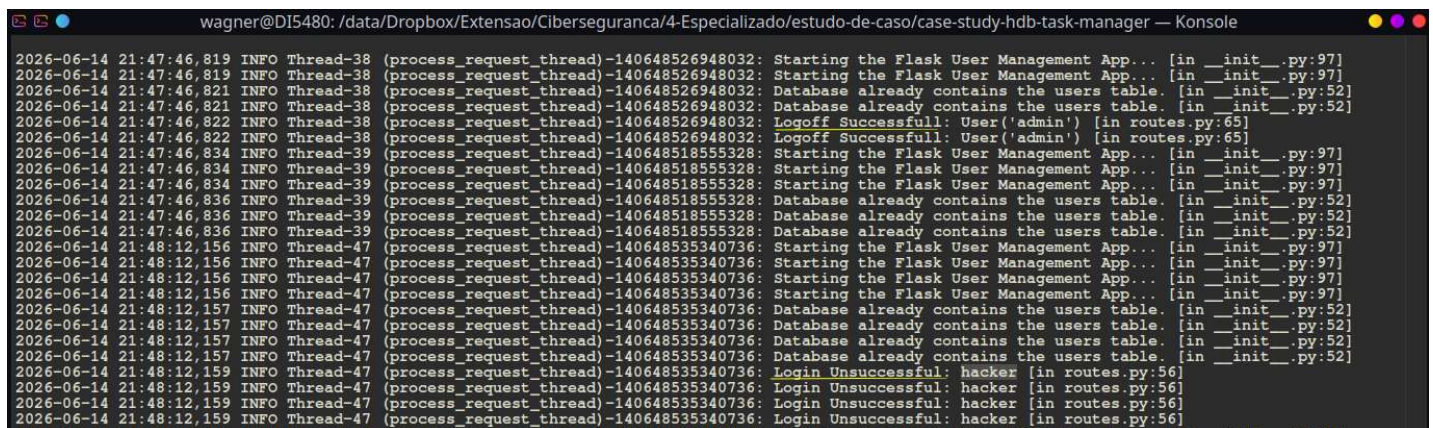
The browser window shows the 'Task Manager' application running at 127.0.0.1:9090/all_tasks. It displays a 'Welcome admin!' message and a 'No Tasks to Display' message with a link to 'Click here to Add Task'.

Melhorias

Os requisitos RF-06 (Pesquisar tarefas) e RF-07 (Login com o google) que ainda não estão contemplados poderiam ser implementados imediatamente ou em sprints seguintes, considerando metodologias ágeis. Como o foco do trabalho é DevOps e DevSecOps, as novas implementações foram deixadas para sprints futuras.

Logs

A aplicação gera log de suas atividades via syslog, bem como sucesso ou fracasso nas autenticações, conforme print abaixo:



```
wagner@DI5480: /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager — Konsole
2026-06-14 21:47:46,819 INFO Thread-38 (process_request_thread)-140648526948032: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:47:46,819 INFO Thread-38 (process_request_thread)-140648526948032: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:47:46,821 INFO Thread-38 (process_request_thread)-140648526948032: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:47:46,821 INFO Thread-38 (process_request_thread)-140648526948032: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:47:46,822 INFO Thread-38 (process_request_thread)-140648526948032: Logoff Successful: User('admin') [in routes.py:65]
2026-06-14 21:47:46,822 INFO Thread-38 (process_request_thread)-140648526948032: Logoff Successful: User('admin') [in routes.py:65]
2026-06-14 21:47:46,834 INFO Thread-39 (process_request_thread)-140648518555328: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:47:46,834 INFO Thread-39 (process_request_thread)-140648518555328: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:47:46,834 INFO Thread-39 (process_request_thread)-140648518555328: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:47:46,836 INFO Thread-39 (process_request_thread)-140648518555328: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:47:46,836 INFO Thread-39 (process_request_thread)-140648518555328: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:47:46,836 INFO Thread-39 (process_request_thread)-140648518555328: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:48:12,156 INFO Thread-47 (process_request_thread)-140648535340736: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:48:12,156 INFO Thread-47 (process_request_thread)-140648535340736: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:48:12,156 INFO Thread-47 (process_request_thread)-140648535340736: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:48:12,156 INFO Thread-47 (process_request_thread)-140648535340736: Starting the Flask User Management App... [in __init__.py:97]
2026-06-14 21:48:12,157 INFO Thread-47 (process_request_thread)-140648535340736: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:48:12,157 INFO Thread-47 (process_request_thread)-140648535340736: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:48:12,157 INFO Thread-47 (process_request_thread)-140648535340736: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:48:12,157 INFO Thread-47 (process_request_thread)-140648535340736: Database already contains the users table. [in __init__.py:52]
2026-06-14 21:48:12,159 INFO Thread-47 (process_request_thread)-140648535340736: Login Unsuccessful: hacker [in routes.py:56]
2026-06-14 21:48:12,159 INFO Thread-47 (process_request_thread)-140648535340736: Login Unsuccessful: hacker [in routes.py:56]
2026-06-14 21:48:12,159 INFO Thread-47 (process_request_thread)-140648535340736: Login Unsuccessful: hacker [in routes.py:56]
2026-06-14 21:48:12,159 INFO Thread-47 (process_request_thread)-140648535340736: Login Unsuccessful: hacker [in routes.py:56]
```


3- Integração contínua e testes automatizados

Utilização do GitHub actions (GHA) para conter problemas de limitação de hardware, principalmente espaço em disco, sem prejuízos de aprendizado pois é muito similar ao Gitlab, usado no curso.

Controle de versão

Controle de versão com o git, repositório disponível [neste link](#). Utilização de branches dev, hom, prod(master).

The screenshot shows the 'Branches' page on GitHub. At the top, there's a navigation bar with 'Overview', 'Yours', 'Active', 'Stale', and 'All' tabs. Below this is a search bar labeled 'Search branches...'. The 'Default' section shows the 'master' branch, updated 3 hours ago, with a 'Default' status and a 'Pull request' button. The 'Your branches' section shows two branches: 'dev' (updated 1 hour ago, 0 behind, 1 ahead) and 'hml' (updated 3 hours ago, 0 behind, 0 ahead). Both have 'Pull request' buttons.

Integração contínua (CI):

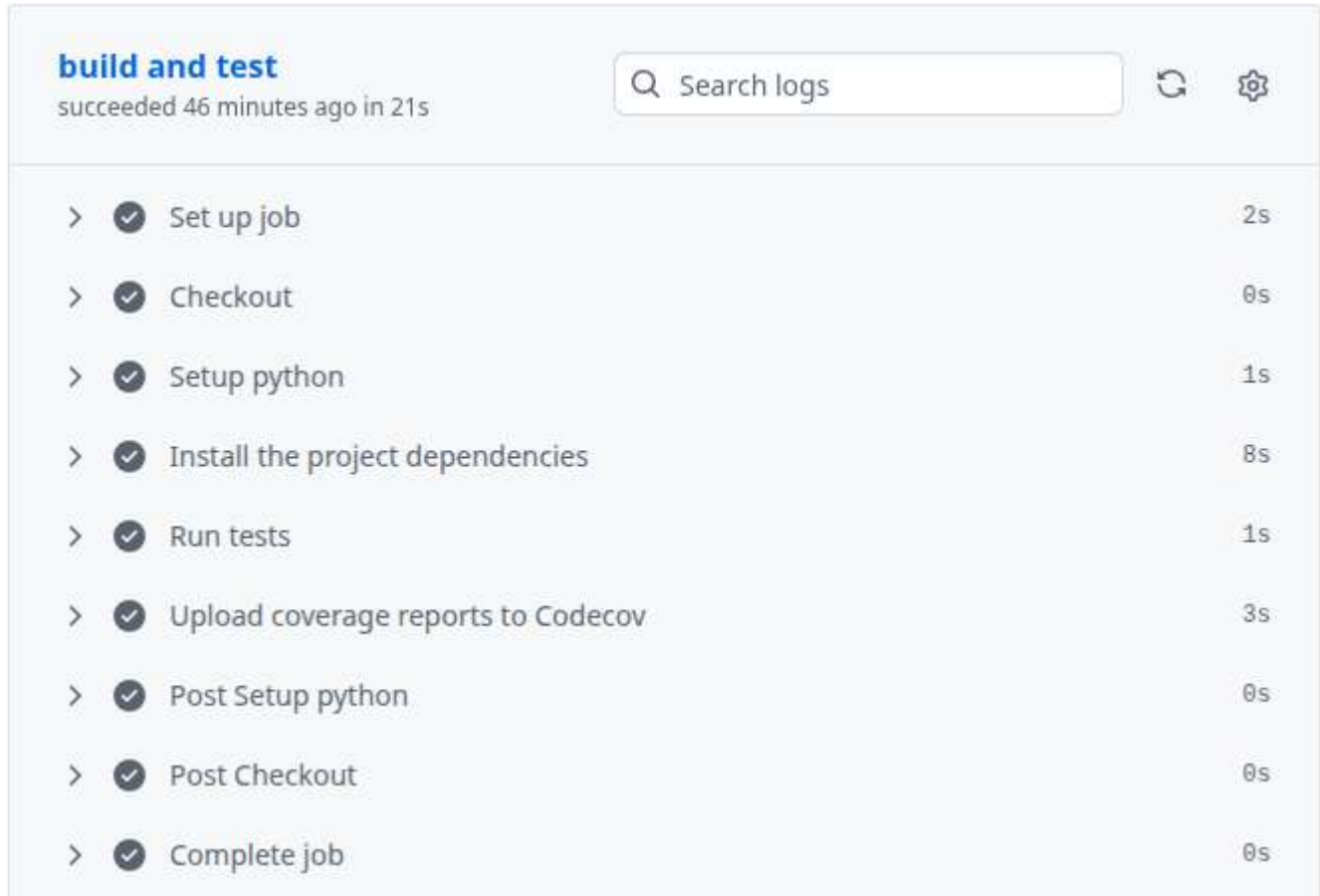
Integração de código diariamente com pequenos commits. A cada commit o código é compilado automaticamente.

The screenshot shows the 'Pulse' page on GitHub for the repository wcpriano/case-study-hdb-task-manager. The page is for the period 'June 5, 2026 - June 12, 2026' with a 'Period: 1 week' dropdown. The 'Overview' section shows '2 Active pull requests' and '0 Active issues'. Below this are four metrics: '2 Merged pull requests', '0 Open pull requests', '0 Closed issues', and '0 New issues'. The 'Summary' section states: 'Excluding merges, 1 author has pushed 20 commits to master and 20 commits to all branches. On master, 47 files have changed and there have been 499 additions and 96 deletions'. The 'Top Committers' section shows a bar chart with '20 commits authored by wcpriano'.

Estes dados ainda estão sendo atualizados, pois ainda serão implementadas as novas fases (SAST, DAST, CD). Mais detalhes e dados atualizados na aba **insights** do repositório, ou diretamente [neste link](#).

Compilação:

A cada commit enviado para o repositório o código é automaticamente compilado para assegurar a consistência do ambiente.

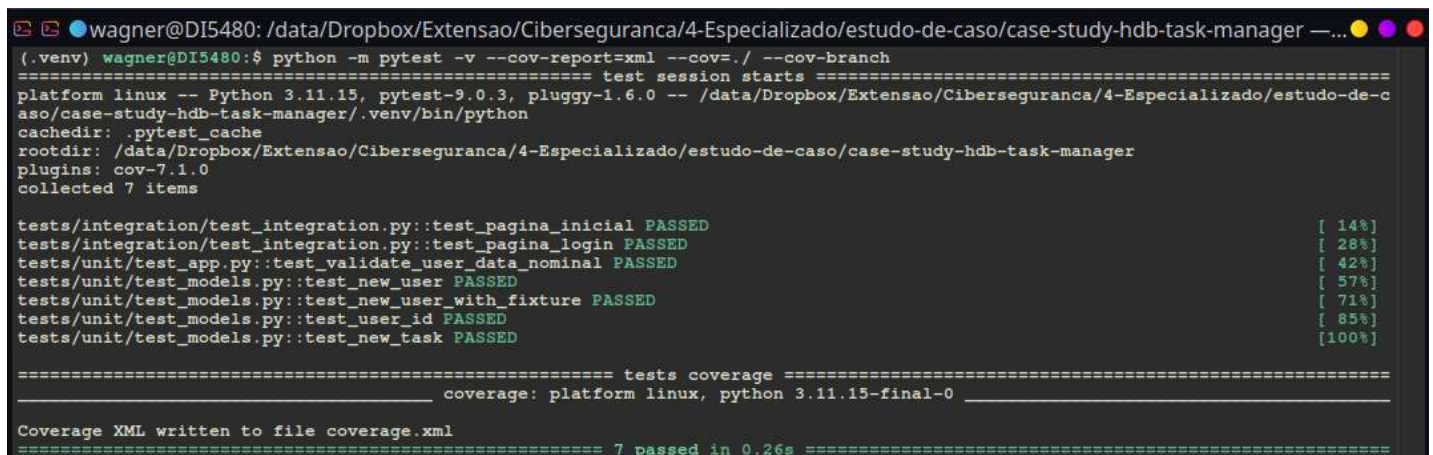


The screenshot shows a CI/CD build log for a project. The title is "build and test" and it indicates the build "succeeded 46 minutes ago in 21s". There is a search bar labeled "Search logs" and a refresh icon. The log consists of a list of steps, each with a checkmark icon, a description, and a duration:

Step	Duration
Set up job	2s
Checkout	0s
Setup python	1s
Install the project dependencies	8s
Run tests	1s
Upload coverage reports to Codecov	3s
Post Setup python	0s
Post Checkout	0s
Complete job	0s

Implementação dos testes automatizados:

Foi feita a implementação dos testes com a biblioteca Pytest e, após as correções, todos os testes passaram, conforme print abaixo, da execução na máquina local:



```
wagner@DI5480: /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager — ...  
(.venv) wagner@DI5480:~$ python -m pytest -v --cov-report=xml --cov=./ --cov-branch  
===== test session starts =====  
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager/.venv/bin/python  
cachedir: .pytest_cache  
rootdir: /data/Dropbox/Extensao/Ciberseguranca/4-Especializado/estudo-de-caso/case-study-hdb-task-manager  
plugins: cov-7.1.0  
collected 7 items  
  
tests/integration/test_integration.py::test_pagina_inicial PASSED [ 14%]  
tests/integration/test_integration.py::test_pagina_login PASSED [ 28%]  
tests/unit/test_app.py::test_validate_user_data_nominal PASSED [ 42%]  
tests/unit/test_models.py::test_new_user PASSED [ 57%]  
tests/unit/test_models.py::test_new_user_with_fixture PASSED [ 71%]  
tests/unit/test_models.py::test_user_id PASSED [ 85%]  
tests/unit/test_models.py::test_new_task PASSED [100%]  
  
===== tests coverage =====  
coverage: platform linux, python 3.11.15-final-0  
  
Coverage XML written to file coverage.xml  
===== 7 passed in 0.26s =====
```

Pipeline CI/CD

A primeira versão do pipeline contempla as seguintes etapas: Checkout, Setup python, Install the project dependencies, Run tests with coverage e Upload coverage reports to Codecov, conforme print abaixo. A pipeline também pode ser acessada diretamente no repositório, por meio [deste link](#).

```
1 # This workflow will test and check quality of the code
2 name: Tests
3 on:
4   push:
5     branches:
6       - dev
7       - dev2
8       - hml
9       - master
10 jobs:
11   build:
12     name: build and test
13     runs-on: ubuntu-latest
14     steps:
15       - name: Checkout
16         uses: actions/checkout@v6
17       - name: Setup python
18         uses: actions/setup-python@v6
19         with:
20           python-version: 3.11
21       - name: Install the project dependencies
22         run: |
23           python -m pip install --upgrade pip
24           pip install -r requirements.txt
25       - name: Run tests with coverage
26         run: python -m pytest -v --cov-report=xml --cov=./ --cov-branch
27       - name: Upload coverage reports to Codecov
28         uses: codecov/codecov-action@v5
29         with:
30           token: ${{ secrets.CODECOV_TOKEN }}
```

Execução do pipeline

A pipeline de CI foi criada e após os commits, o build é feito automaticamente e o testes foram integrados para executarem em seguida do build:

The screenshot displays the GitHub Actions interface for the repository 'wcipriano / case-study-hdb-task-manager'. The 'Actions' tab is selected, showing a list of workflow runs for the 'Tests' workflow (code-quality-test.yml). The interface includes a search bar, filters for workflow runs, and a table of recent runs.

Event	Status	Branch	Actor	Time
Merge pull request #2 from wcipriano/hml	Success	master	wcipriano	1 hour ago
Merge pull request #1 from wcipriano/dev2	Success	hml	wcipriano	Today at 8:52 AM
instance folder to log files	Success	dev2	wcipriano	Today at 8:39 AM

Todos os testes são executados automaticamente e na execução abaixo todos passaram com sucesso, incluindo os testes unitários, de integração e end-to-end.

Todos os detalhes da execução podem ser visualizados no repositório, neste [link direto](#).

build and test

succeeded 5 minutes ago in 21s

Search logs

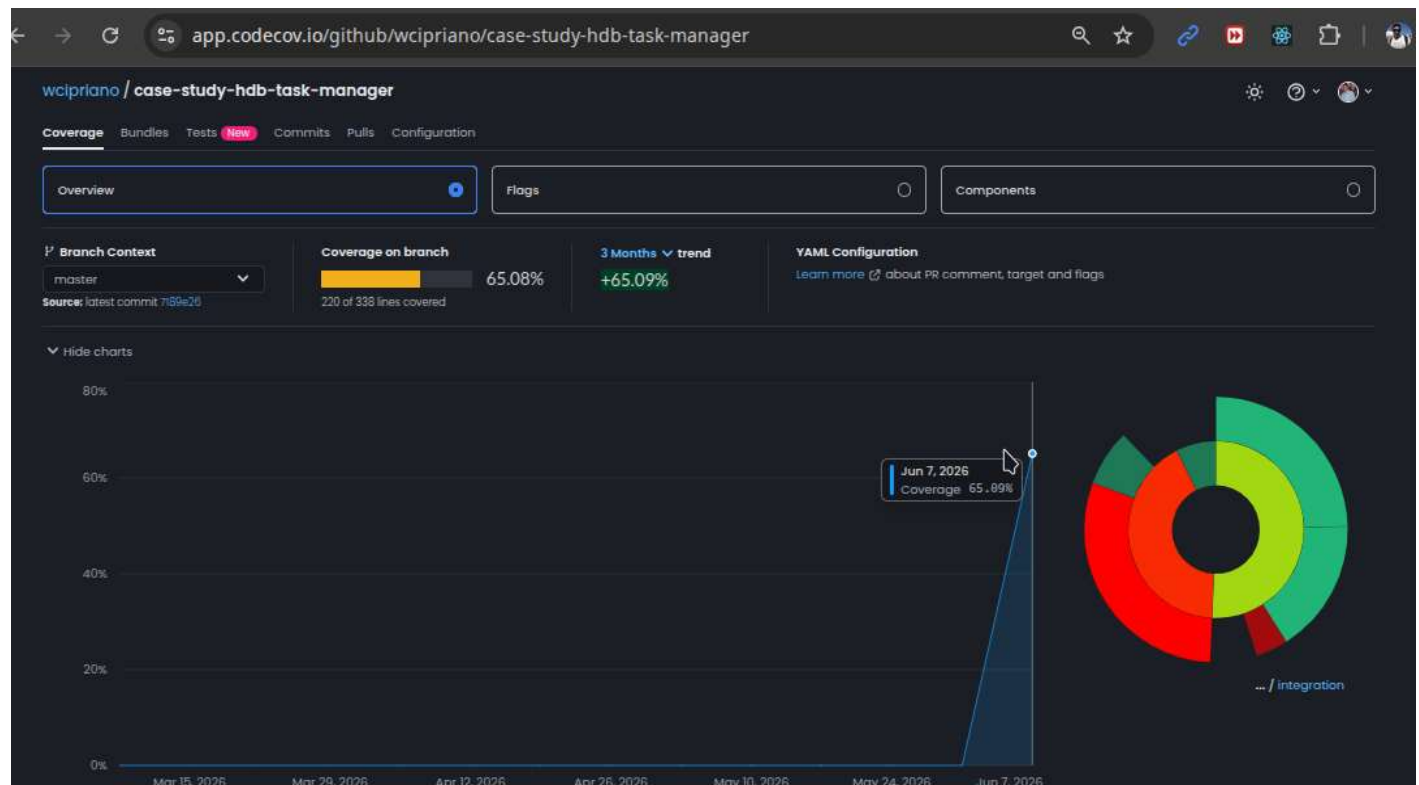
1s

Run tests

18 tests/integration/test_integration.py::test_pagina_inicial PASSED [14%]
19 tests/integration/test_integration.py::test_pagina_login PASSED [28%]
20 tests/unit/test_app.py::test_validate_user_data_nominal PASSED [42%]
21 tests/unit/test_models.py::test_new_user PASSED [57%]
22 tests/unit/test_models.py::test_new_user_with_fixture PASSED [71%]
23 tests/unit/test_models.py::test_user_id PASSED [85%]
24 tests/unit/test_models.py::test_new_task PASSED [100%]
25
26 ===== tests coverage =====
27 _____ coverage: platform linux, python 3.11.15-final-0 _____
28
29 Coverage XML written to file coverage.xml
30 ===== 7 passed in 0.23s =====

Cobertura de testes

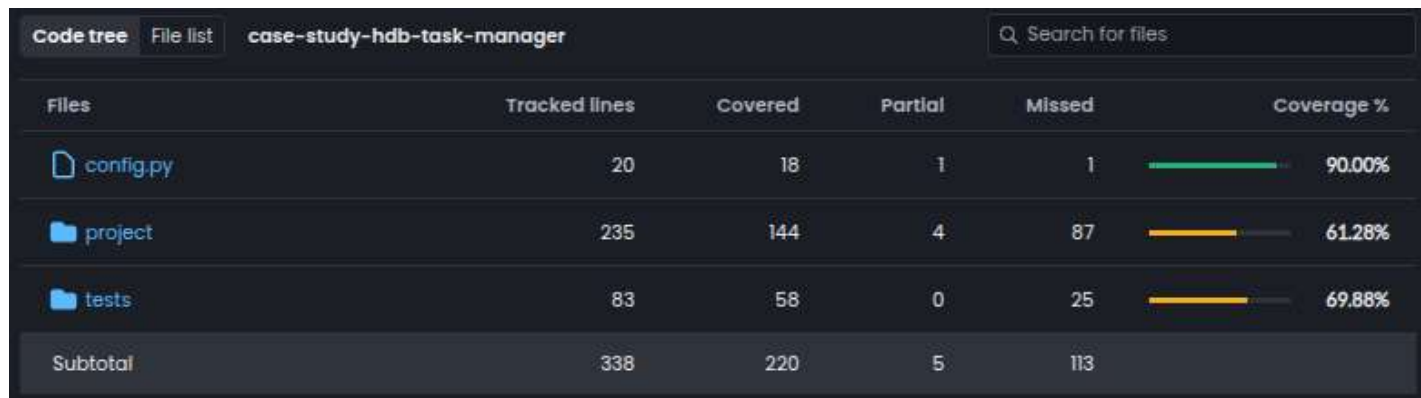
Conforme pipeline exibida acima, no passo "Run tests with coverage" os testes são executados e os dados sobre a cobertura são gerados em formato XML (`--cov-report=xml`). No passo seguinte (Upload coverage reports to Codecov) os dados são enviados automaticamente para a plataforma CodeCov (<https://app.codecov.io/>). A partir do envio destes dados, esta plataforma exibe vários dados sobre a cobertura de testes do repositório, incluindo um gráfico que mostra o histórico e o percentual de cobertura ao longo do tempo.



Nesta primeira versão a cobertura de testes está em 65%. A medida que novos testes são criados e executados, os dados são atualizados na plataforma, permitindo visualizar a evolução.

Não existe um numero obrigatório, absoluto para o percentual de cobertura de testes, mas uma indicação comum é de que seja uma faixa entre 80% e 90%. O foco deve estar nos caminhos críticos e regras de negócio, ou seja, na qualidade dos testes e não na quantidade. Buscar 100% de cobertura frequentemente gera retornos decrescentes, tornando o processo lento e caro sem garantir um software livre de falhas.

Outra informação importante que o CodeCov disponibiliza é o percentual de cobertura em cada pasta ou arquivo do código-fonte, conforme imagem abaixo. Deste modo novos testes serão direcionados para os pontos mais deficitários.



The screenshot shows the CodeCov interface for a repository named 'case-study-hdb-task-manager'. It displays a table with columns for 'Files', 'Tracked lines', 'Covered', 'Partial', 'Missed', and 'Coverage %'. The table lists three files: 'config.py' (90.00% coverage), 'project' (61.28% coverage), and 'tests' (69.88% coverage). A 'Subtotal' row at the bottom shows 338 tracked lines, 220 covered, 5 partial, and 113 missed lines.

Files	Tracked lines	Covered	Partial	Missed	Coverage %
 config.py	20	18	1	1	90.00%
 project	235	144	4	87	61.28%
 tests	83	58	0	25	69.88%
Subtotal	338	220	5	113	

Mais detalhes e os percentuais de cobertura atualizados poderão ser consultados diretamente [neste link](#).

4- Análise Estática de Código (SAST)

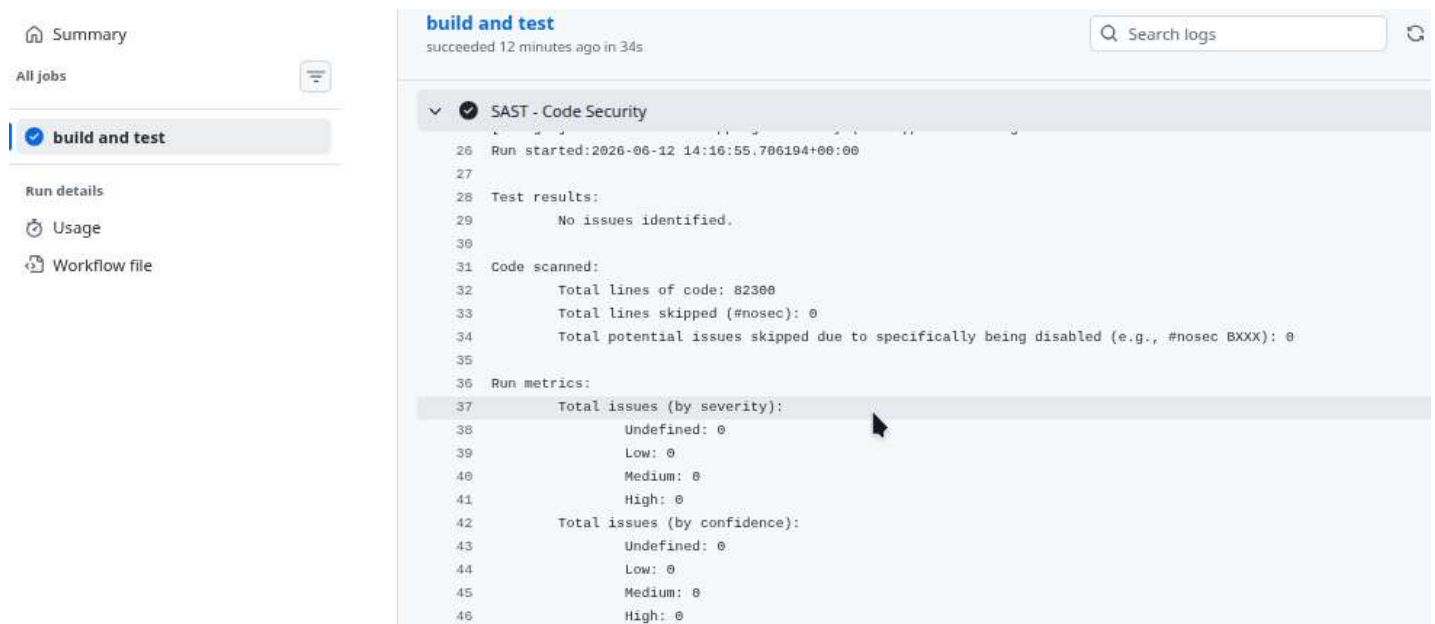
Várias ferramentas foram testadas e duas foram integradas no pipeline, uma para análise de segurança de código e outra focada na validação das dependências utilizadas do projeto, conforme detalhado a seguir.

Análise estática de segurança

Para a análise estática, foi utilizada a ferramenta **Bandit**, de análise de segurança em Python. Esta ferramenta analisa o código em busca de problemas de segurança comuns em códigos Python.

Após finalizado o processo a ferramenta gera um relatório com o resumo do processo, contendo total de linhas escaneadas, total de problemas encontrados, etc.

O processo executou sem problemas tanto no ambiente local, com o docker, quanto na pipeline do GHA, conforme prints a seguir.



O resultado completo pode ser visualizado [neste link](#).

Neste caso não teve nenhum alerta de segurança pois o código está muito atualizado, mas é mais comum termos alguns alertas sendo exibidos, principalmente por problemas com dependências.

Mais informações sobre o Bandit podem ser obtidas no Python Package Index: <https://pypi.org/project/bandit/>.

Análise de dependências

Para a análise de dependências, para identificar bibliotecas desatualizadas ou com falhas de segurança, foram testadas algumas ferramentas, incluindo a dependency-check e a pip-audit. Essa foi integrada no pipeline com sucesso.

O pip-audit é um scanner de vulnerabilidades dedicado ao Python, que usa como entrada um arquivo que possui lista de dependências do projeto (requirements.txt) ou um virtual env (venv).

Conforme pode ser visto no print abaixo, o pip-audit não encontrou nenhuma vulnerabilidade conhecida nas dependências utilizadas no projeto.

✓ change pip-audit to use gh-action #20

Summary

All jobs

build and test

Run details

Usage

Workflow file

Triggered via push 1 minute ago

Status

Total duration

Artifacts

wcipriano pushed → c7f4678 dev

Success

36s

-

code-quality-test.yml

on: push

build and test 32s

build and test summary

pip-audit exited successfully

Raw pip-audit output

No known vulnerabilities found

O resultado completo pode ser visualizado [neste link](#).

O GH também disponibiliza, de modo built-in a ferramenta Dependabot que valida as dependencias do projeto e inclusive avisa, por email ao desenvolvedor sobre problemas de segurança encontrados nos pacotes utilizados. Eu utilizo muito esta ferramenta em outro repositório que sou manetenedor ([pretty-print-confusion-matrix](#)) para manter o software sempre atualizado e mais seguro possível.

Não basta executar ferramentas de análise, é necessário corrigir os problemas encontrados. Algumas ferramentas disponibilizam correções automáticas, mas sempre será necessária uma revisão e testes do analista responsável. No nosso caso, como não foi encontrado nenhum problema, vamos avançar para a proxima fase, análise dinâmica (DAST)

5- Análise Dinâmica de Segurança (DAST)

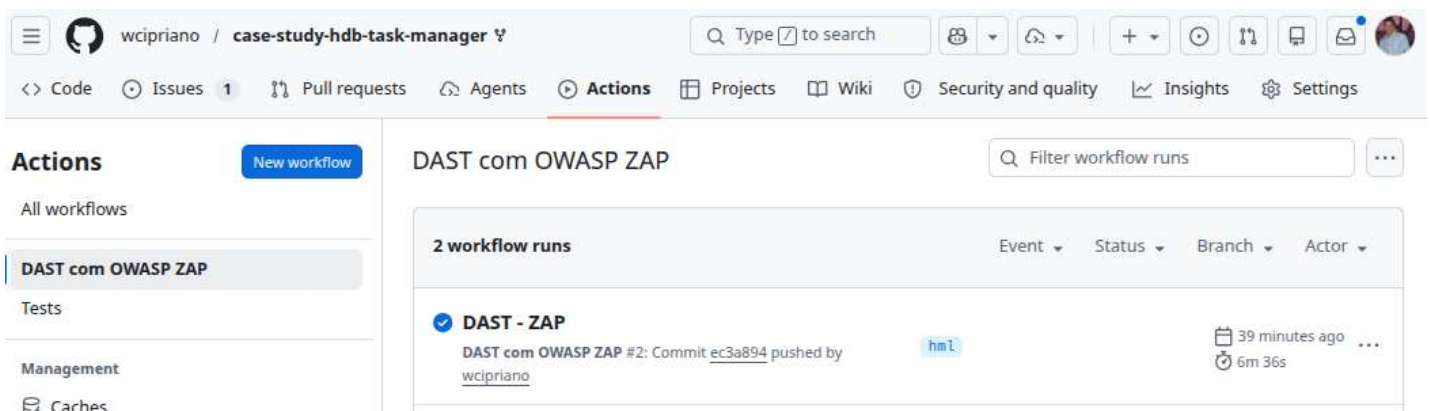
Para a análise dinâmica, foi utilizado o **OAWSP ZAP**, em execução em um container Docker. O Zed Attack Proxy (ZAP) é uma ferramenta gratuita de testes de penetração que busca vulnerabilidades em aplicações web, a partir de uma imagem docker.

A ferramenta foi integrada em uma pipeline do GHA (dast.yml) para fazer os testes de segurança contra a aplicação em execução. A pipeline faz o setup e inicia a aplicação em um container, deixando a aplicação rodando na porta 8088, a seguir por meio da execução em outro container docker (via GHA) a ferramenta ZAP faz as requisições contra a aplicação e gera o relatório de forma automatizada.

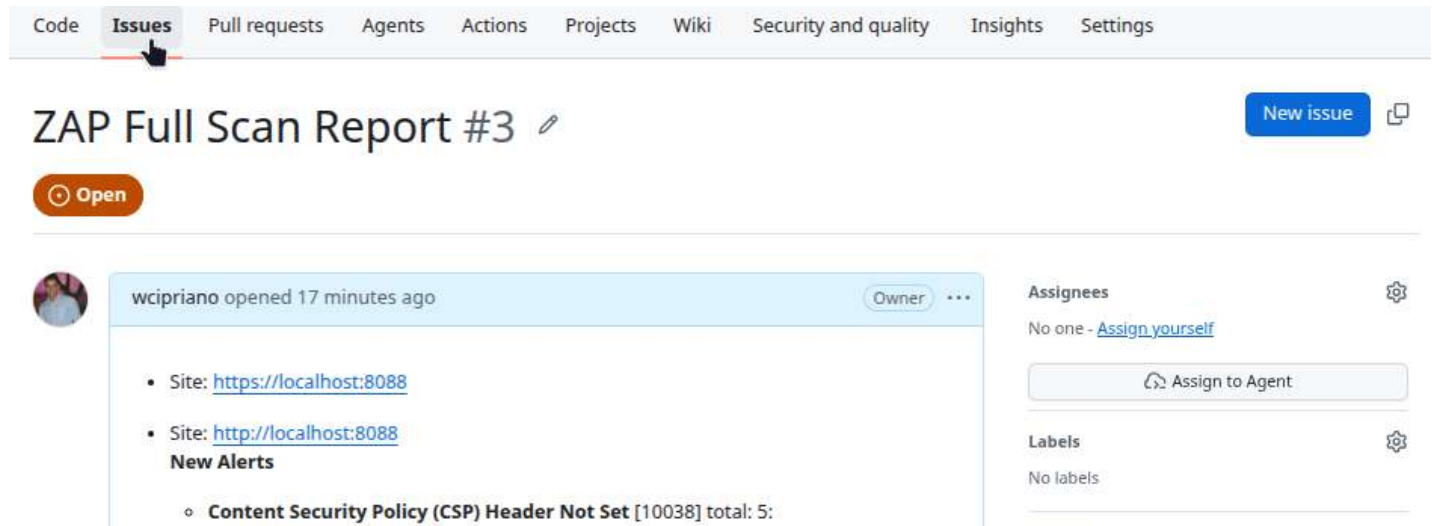
Esta pipeline contempla os seguintes passos: (1)Checkout, (2)Setup python, (3)Install the project dependencies (and run app), (4)OWASP ZAP Full Scan, (5)Save ZAP Report, conforme print abaixo. O código da pipeline também pode ser acessada diretamente no repositório, por meio [deste link](#).

```
1 name: DAST com OWASP ZAP
2 on:
3   push:
4     branches: [master, hml]
5   pull_request:
6     branches: [master, hml]
7 jobs:
8   dast-scan:
9     name: build run and test ZAP
10    runs-on: ubuntu-latest
11
12    steps:
13      - name: Checkout
14        uses: actions/checkout@v6
15
16      - name: Setup python
17        uses: actions/setup-python@v6
18        with:
19          python-version: 3.11
20
21      - name: Install the project dependencies
22        run: |
23          python -m pip install --upgrade pip
24          pip install -r requirements.txt
25          flask --app ${{vars.FLASK_APP}} run --port ${{vars.FLASK_RUN_PORT}} &
26          sleep 5
27
28      - name: OWASP ZAP Full Scan
29        uses: zapproxy/action-full-scan@v0.13.0
30        with:
31          target: 'http://localhost:${{vars.FLASK_RUN_PORT}}'
32          rules_file_name: '.zap/rules.tsv'
33          token: ${{ secrets.HDB_ACCESS_TOKEN }}
34
35      - name: Save ZAP Report
36        uses: actions/upload-artifact@v7
37        with:
38          name: zap-report
39          path: |
40            **/zap-report-html.html
41            **/zap-report-json.json
```

A execução bem sucedida pode ser validada no print abaixo ou diretamente [neste link](#) (do GH Actions).



No step 5 foi feita a configuração do token (HDB_ACESS_TOKEN) de acesso à API do GH e desta forma, a ferramenta integra com o GH, via API REST, e cria automaticamente uma **issue com o relatório dos problemas** e alertas encontrados no escaneamento (conforme abaixo), uma automação muito útil. No print abaixo pode ser visualizada a issue criada automaticamente pela ferramenta.



The screenshot shows a GitHub repository interface. At the top, there's a navigation bar with tabs: Code, **Issues** (selected), Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. Below the navigation bar, the issue title 'ZAP Full Scan Report #3' is displayed with a pencil icon for editing. To the right of the title is a 'New issue' button. Below the title is an 'Open' button. The issue content area shows a user profile picture, the text 'wcpiriano opened 17 minutes ago', and a list of sites scanned: 'https://localhost:8088' and 'http://localhost:8088'. Below the sites, it says 'New Alerts' and lists 'Content Security Policy (CSP) Header Not Set [10038] total: 5:'. To the right of the issue content, there are sections for 'Assignees' (No one - Assign yourself) and 'Labels' (No labels).

O relatório completo do ZAP pode ser acessado por meio [deste link direto](#). Não foi encontrado nenhum problema crítico, mas alguns alertas de segurança referente à Cookies e informações acessíveis que podem ser ocultadas. Os principais alertas encontrados foram:

- Content Security Policy (CSP) Header Not Set
- HTTP Only Site
- Missing Anti-clickjacking Header
- Vulnerable JS Library
- Cookie Slack Detector
- Cookie without SameSite Attribute
- Cross-Origin-Embedder-Policy Header Missing or Invalid
- Cross-Origin-Opener-Policy Header Missing or Invalid
- Cross-Origin-Resource-Policy Header Missing or Invalid
- Permissions Policy Header Not Set
- Server Leaks Version Information via "Server" HTTP Response Header Field
- X-Content-Type-Options Header Missing
- Authentication Request Identified
- Information Disclosure - Suspicious Comments

Como não existem correções urgentes, os alertas serão analisados e corrigidos posteriormente, o status pode ser acompanhado na [issue que foi criada](#).

6- Entrega Contínua (CD)

Para cada merge request aberto na branch principal, o deploy é realizado automaticamente com a criação de um ambiente de stage (homologação) para avaliação manual. Assim, as alterações propostas podem ser testadas em uma instância temporária independente antes de subir para produção e antes mesmo de serem aceitas na branch principal (master). Este ambiente é usado para simular o ambiente de produção, para aprovação do responsável, antes do deploy definitivo para PRD.

Os testes dinâmicos de segurança de aplicação (DAST) também são executados neste ambiente com a utilização da ferramenta OWASP ZAP.

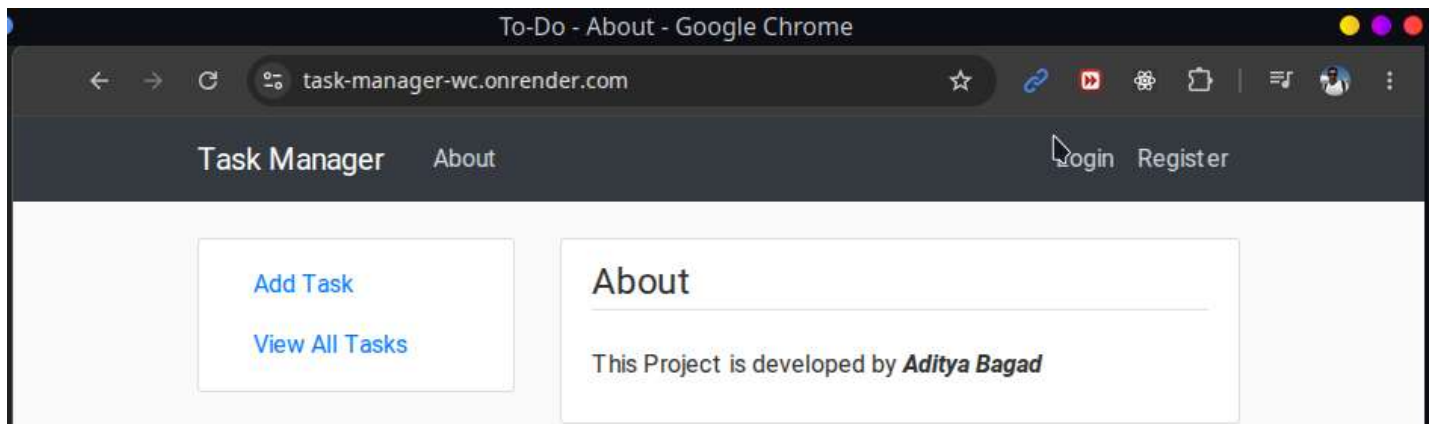
A ferramenta utilizada foi a "Pull request previews" do Render(render.com). Ela cria um ambiente de pré-visualização (homologação) para solicitações de pull abertas em uma branch vinculada, conforme [a documentação](#). A seguir está o relato de como foi feita a configuração e os testes.

Depois de logado, no dashboard foi feita a inclusão de um serviço (Web Service) chamado task-manager-wc e este serviço foi integrado com o [repositório](#) do GH, conforme configurações abaixo:

Name	task-manager-wc
Source	https://github.com/wcipriano/case-study-hdb-task-manager
Branch	Master
Build Command	Build Command
Start Command	flask --app \$FLASK_APP run --host \$HOST --port \$PORT

Após esta configuração inicial e o deploy executado, o ambiente foi disponibilizado online em: <https://task-manager-wc.onrender.com>, conforme print abaixo.

Cabe ressaltar que se trata de uma **hospedagem gratuita** e as instâncias gratuitas são desativadas após períodos de inatividade, desta forma, no primeiro acesso, vai demorar alguns segundos para carregar a página do sistema de gerenciamento de tarefas, mas depois fica normal a navegação.



A seguir foi feita a configuração da ferramenta Pull request previews (PR Previews: Automatic) para que, a cada pull request (PR) aberto na branch main, fosse disponibilizado um ambiente temporário independente e online para validação com o endereço seguindo a seguinte estrutura:

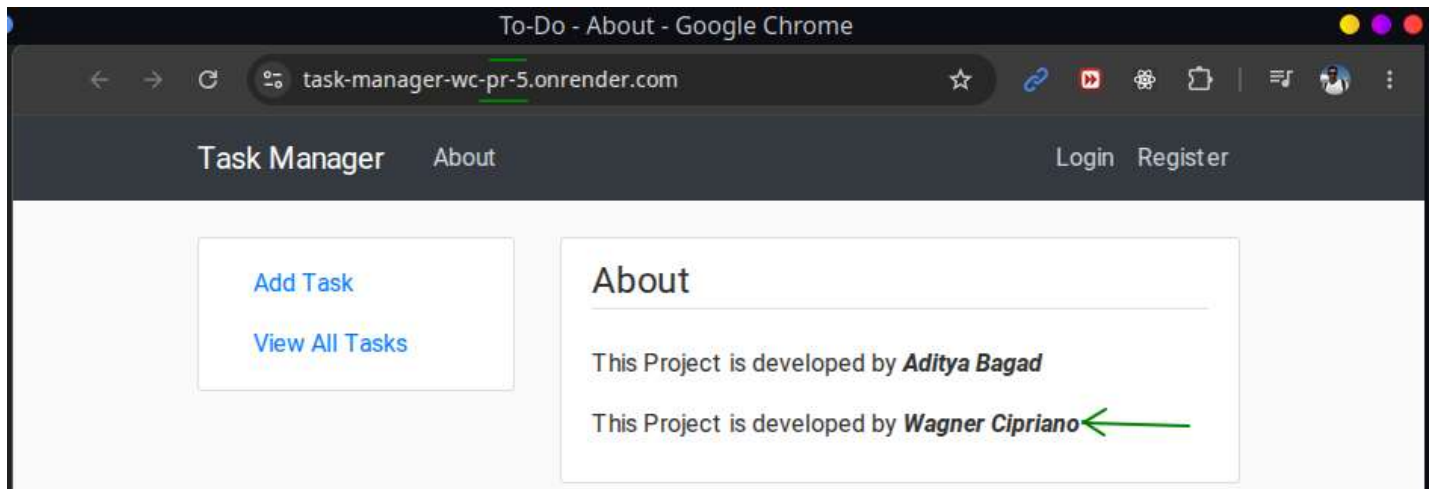
Lógica: `<protocol><subdomain>-pr-<pr_number>.<host>`
Exemplo: `https://task-manager-wc-pr-5.onrender.com/`

Finalizadas as configurações, iniciou-se os testes e a validação do fluxo. Para isso foi feita uma alteração no código, inserindo na página sobre (about), incluindo o nome do segundo autor (Wagner Cipriano). Em seguida foi feito o commit na branch dev e o merge para a branch hml.

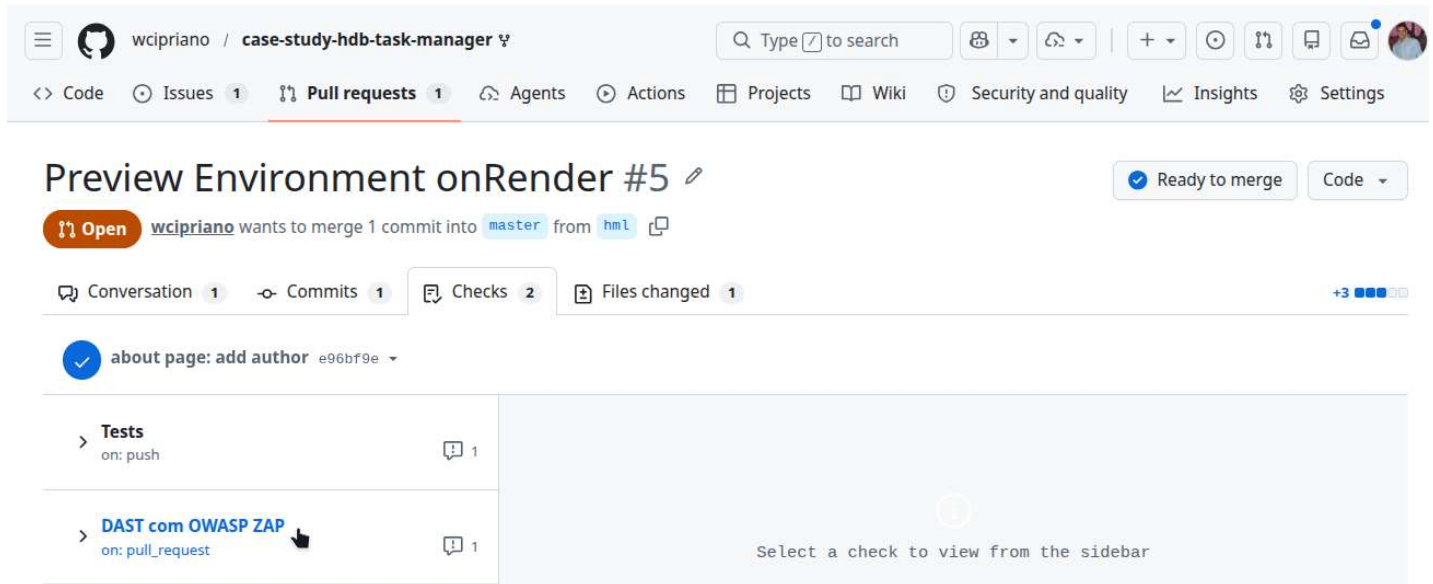
Em seguida foi feita a criação de um PR (de número 5) na branch master a partir da hml.

Após concluído o deploy do ambiente de homologação, a validação foi feita com sucesso, conforme print abaixo, com o nome do novo autor sendo inserido corretamente na página.

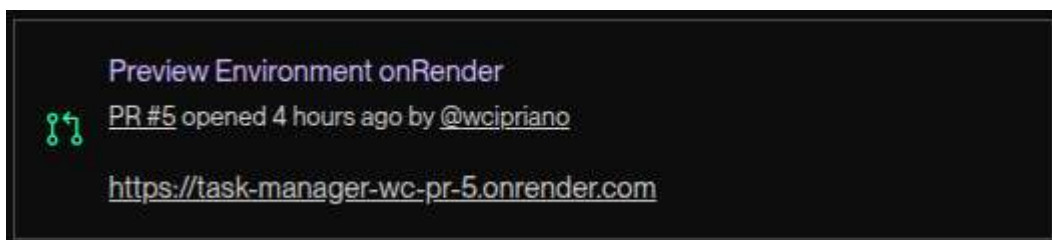
Esta automação é muito útil, pois o ambiente PRD simulado é gerado e disponibilizado antes de serem aprovadas as alterações na branch main.



Este evento também disparou os testes testes dinâmicos de segurança de aplicação , conforme print abaixo.



No Dashboard do render é possível verificar que existe um ambiente de preview, vinculado com o PR 5, conforme print abaixo:



A integração funciona no ciclo completo da solicitação de alteração, pois quando o PR é concluído (mergeado), o render remove automaticamente o ambiente provisionado, o que é importante para economizar recursos.

No PR, no GH. também é possível ver a informação do deploy do render, assim como o link para acessar diretamente o ambiente de homolog, conforme print abaixo, tudo integrado, de forma simples e funcional:

The screenshot displays a GitHub Pull Request (PR) #5 titled "Preview Environment onRender #5". The interface includes a top bar with a "Ready to merge" status and a "Code" dropdown. Below the title, a notification states "wcpriano wants to merge 1 commit into master from hml". The main content area shows a conversation with a comment from "wcpriano" about adding an author to the "about" page. A deployment status is shown as "wcpriano deployed to hml - task-manager-wc PR #5 3 hours ago — with Render". A "View deployment" button is visible next to the deployment information. The right sidebar contains sections for "Reviewers" (listing "Copilot"), "Assignees" (none), "Labels" (none), "Projects" (none), "Milestone" (none), and "Development" (noting that merging may close issues). At the bottom, a message states "This branch was successfully deployed" with "1 active deployment". A deployment summary at the very bottom shows "hml - task-manager-wc PR #5 — e96bf9e6 Deployed 3 hours ago by wcpriano" with a "View deployment" button.

Todos os detalhes do PR 5 podem ser acessados diretamente [neste link](#).

7- Feedback e Monitoramento

Nesta fase foi feita a instalação de ferramentas que monitoram a aplicação em produção para detectar possíveis problemas. Foram utilizadas as ferramentas Prometheus e Grafana por serem soluções consagradas e amplamente utilizadas, além de ser a solução que a equipe possui mais experiência, a pesar de existirem outras boas ferramentas de monitoramento como Wazuh(Foco em Segurança de Endpoints e SIEM), Falco(Foco em Runtime Security para Contêineres), Datadog(Foco em Observabilidade de Nuvem).

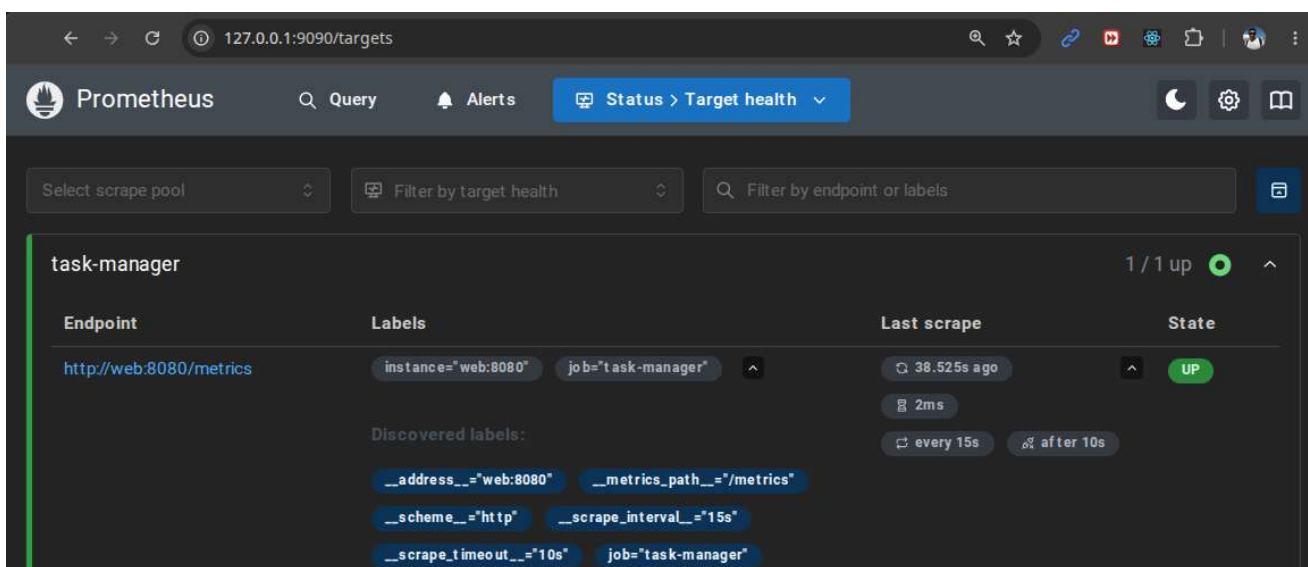
Containers Docker

O monitoramento foi incluído na pipeline assim como a instalação das ferramentas que monitoram a aplicação em produção. Abaixo está a versão final do arquivo docker compose com os containers Prometheus e Grafana que também pode ser visualizada [neste link](#).

```
1 version: '3.9'
2 services:
3   web:
4     restart: on-failure
5     build:
6       context: .
7       dockerfile: Dockerfile
8     image: task-manager
9     container_name: task-manager
10    ports:
11      - ${PORT:-5000}:${PORT:-5000}
12    volumes:
13      - taskdb:/app/instance
14    environment:
15      - FLASK_APP=${FLASK_APP:-APP}
16      - FLASK_ENV=${FLASK_ENV:-development}
17      - FLASK_DEBUG=${FLASK_DEBUG:-1}
18      - PORT=${PORT:-5000}
19    env_file:
20      - .env
21    networks:
22      - web
23
24   prometheus:
25     image: prom/prometheus
26     container_name: tm_prometheus
27     restart: always
28     volumes:
29       - ./prometheus:/etc/prometheus
30       - ./prometheus/data:/prometheus
31     ports:
32       - 9090:9090
33     networks:
34       - web
35
36   grafana:
37     image: grafana/grafana
38     container_name: tm_grafana
39     user: "472:472"
40     restart: always
41     environment:
42       GF_INSTALL_PLUGINS: 'grafana-clock-panel,grafana-simple-json-datasource'
43       GF_SECURITY_ADMIN_USER: 'admin'
44       GF_SECURITY_ADMIN_PASSWORD: 'admin'
45       GF_USERS_ALLOW_SIGN_UP: 'false'
46     volumes:
47       - ./grafana:/var/lib/grafana
48     ports:
49       - 3000:3000
50     depends_on:
51       - prometheus
52     networks:
53       - web
54
55 networks:
56   name: web-network
```

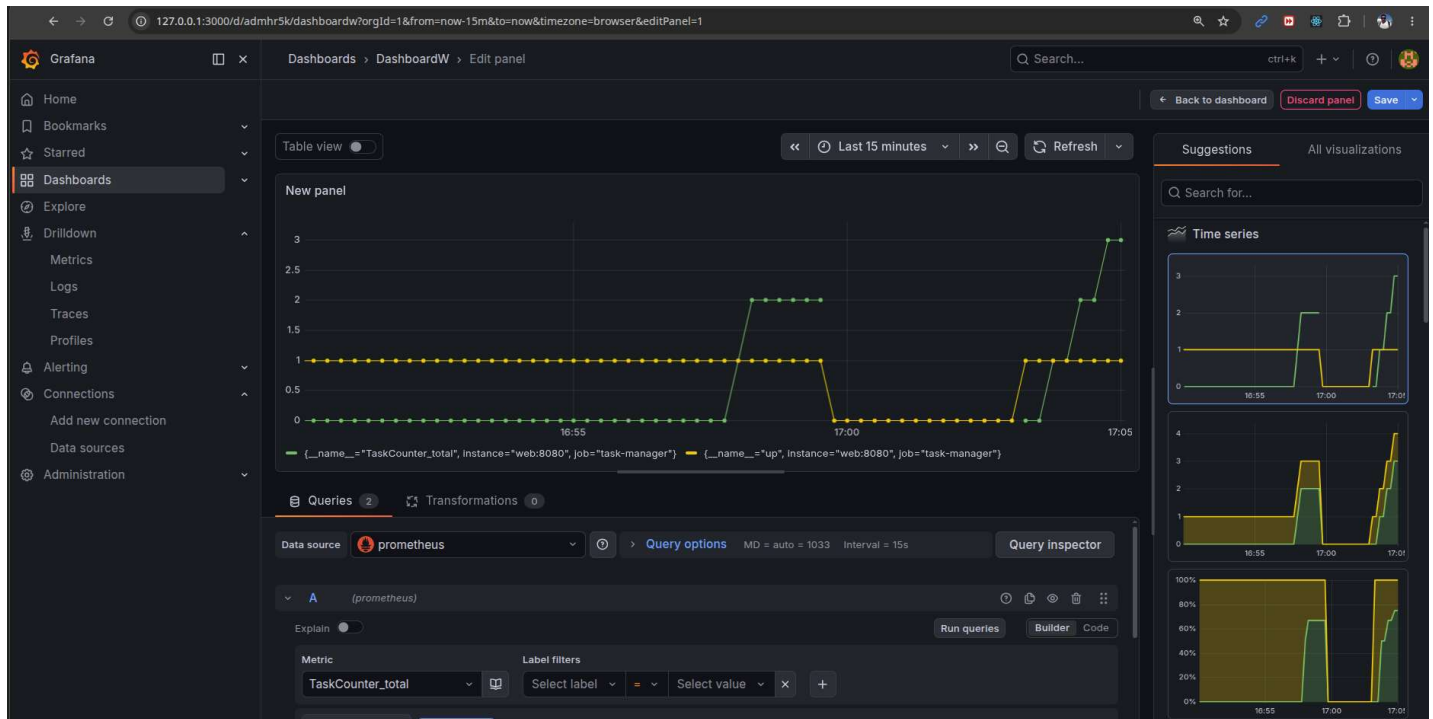
Prometheus

O serviço do Prometheus estava apresentando erro ao tentar conectar com a aplicação (connection refused) mesmo com a aplicação rodando, pois alvo estava apontando para "127.0.0.1:8080", o endereço que a aplicação estava respondendo normalmente no host, mas não pelo container. Nas configurações do Prometheus (prometheus.yml) foi necessária a alteração do alvo para a rede do docker-compose (web) para funcionar (web:8080). A seguir o print da tela do Prometheus mostrando o serviço em funcionamento e o endpoint da aplicação configurado corretamente com o status ativo.



Grafana

Abaixo o print da tela do Grafana com todas as configurações concluídas, com um painel exibindo o monitoramento da métrica TaskCounter e o status da aplicação task-manager (1 para ativo e 0 para inativo).



A versão final dos arquivos de configuração do Prometheus e do Grafana (datasource Prometheus) estão disponíveis abaixo:

```
prometheus.yml
1 global:
2   scrape_interval: 15s
3
4   scrape_configs:
5     - job_name: 'task-manager'
6       static_configs:
7         - targets: ['web:8080']
```

```
Current version grafana-provisioning/datasources/prometheus.yml
1 apiVersion: 1
2
3 deleteDatasources:
4   - name: prometheus
5     orgId: 1
6
7 datasources:
8   - name: prometheus
9     uid: tm-prometheus
10    type: prometheus
11    access: proxy
12    url: http://prometheus:9090
13    isDefault: true
14    editable: true
```

Implementações

Foi feita a criação dos containers Prometheus e Grafana, para ativação dos serviços; instalação da biblioteca prometheus-client para envio das métricas da aplicação para o Prometheus; modificação da aplicação para disponibilizar o número de tarefas criadas (TaskCounter) como uma métrica a ser monitorada; disponibilizada a rota "/metrics" na aplicação, para alimentar o Prometheus; configuração do Prometheus para leitura das metrcias da aplicação (static_configs.targets); e configuração do Grafana para criar o dashboard/painel de visualização das métricas.

8- Conclusão

A seguir, um fechamento sobre resultados, comentários sobre as ferramentas e os trabalhos futuros.

Ferramentas

A principal ferramenta utilizada foi o **Github actions**. É uma ótima ferramenta para automatizar os fluxos de desenvolvimento de software. Foi a opção natural por já ter mais conhecimento e por não ter limitação de hardware (caso de repo público). Ela conta com um marketplace que possui várias extensões para automatizar os workflows, o que reduz a necessidade de escrever manualmente, padroniza o processo e aumenta a confiabilidade das etapas da pipeline, visto que são ferramentas amplamente utilizadas e muito estáveis..

Resultados alcançados

Todo o processo de desenvolvimento, implementação e segurança do sistema foi feito seguindo padrões e ferramentas mais modernas do mercado, ainda que em níveis gratuitos, para teste, e os principais resultados alcançados foram:

- **Segurança Integrada desde o Primeiro Commit (Shift-left):** Vulnerabilidades e falhas de configuração são identificadas e corrigidas antes de chegarem ao ambiente de produção
- **Redução do Tempo de Correção (MTTR):** Uso de ferramentas automatizadas no fluxo CI/CD.
- **Conformidade Contínua:** Auditorias de segurança e conformidade incorporadas na esteira de deploy.

Lições aprendidas

É necessário parar um pouco a rotina do dia a dia e conhecer melhor as ferramentas disponíveis para automatizar tarefas, trazer qualidade e segurança para os nossos códigos. Na correria do dia a dia nem sempre temos essa oportunidade, e este trabalho proporciona esta boa experiência.

Nem todos os erros apontados pelas ferramentas é possível ou necessário corrigir no momento, pode ser tratado como débito técnico para futuras correções. Por exemplo, na etapa de DAST, a ferramenta Zed Attack Proxy (ZAP) gerou vários alertas de segurança referente à Cookies e informações técnicas acessíveis, que não são urgentes, nem são possíveis de serem corrigidos de forma imediata, depende de um planejamento.

Trabalhos futuros

Algumas oportunidade para continuidade do trabalho foram identificadas e serão elencadas a seguir.

Melhorias

Implementar os requisitos que atualmente não são atendidos pelo software, como RF-06 e RF-07.

Correção dos erros de segurança

Analisar com mais detalhe e buscar soluções para os erros encontrados no repositório, como forma de aprendizado e melhoria contínua.

Infrastructure as Code

Implementar os scripts de configuração para provisionamento automático da infra, utilizando o Terraform.